Gate 04 — Django Admin Configuration

Status: Complete **Preconditions:** Gate 03 complete **Unlocks:** Gate 06 (IUCN Sync & Seed Data) — coordinators can begin data entry after this gate

---

Purpose

Configure Django Admin so that conservation coordinators can manage all MVP data without a custom frontend. At MVP, Django Admin is the complete coordinator UI. This gate specifies Admin registration — list_display, list_filter, search_fields, and inline models. No custom views, no custom actions.

Deliverables

- All MVP models registered in Django Admin with configuration specified below
- Tier 5 admins can manage all data; Tier 3–4 users restricted to records for their affiliated institution (enforced via `get_queryset()` overrides on ModelAdmin subclasses)
- Admin branding: site header "Madagascar Freshwater Fish Conservation Platform", site title "MFFCP Admin"

Admin Registrations

`species.Species`

```
list_display = [
    "scientific_name", "taxonomic_status", "family", "genus",
    "endemic_status", "iucn_status", "cares_status", "shoal_priority",
]
list_filter = [
    "taxonomic_status", "iucn_status", "cares_status",
    "endemic_status", "family", "shoal_priority",
]
search_fields = [
    "scientific_name", "provisional_name",
    "commonname__name",  # search across CommonName inline
]
readonly_fields = ["created_at", "updated_at"]
inlines = [ConservationAssessmentInline, CommonNameInline, SpeciesLocalityInline]
```

`ConservationAssessmentInline` (TabularInline):

```
model = ConservationAssessment
extra = 1
fields = [
    "category", "source", "review_status", "criteria",
    "assessor", "assessment_date", "review_notes", "flagged_by", "flagged_date",
"notes",
]
readonly_fields = ["flagged_date"]
```

`CommonNameInline` (TabularInline):

```
model = CommonName
extra = 1
fields = ["name", "language", "is_preferred"]
```

User stories:

As a Tier 5 administrator entering a new undescribed morphospecies, **I want** the Species admin form to accept null values for `authority` and `year_described` when `taxonomic_status = 'undescribed_morphospecies'`, **so that** I can create records for species without formal description.

Acceptance Criteria:

Given a Tier 5 admin creating a Species record with `taxonomic_status = 'undescribed_morphospecies'` **When** `authority` and `year_described` are left blank **Then** the record saves without validation errors

Given a Tier 5 admin filtering the Species list by `taxonomic_status = 'undescribed_morphospecies'` **When** the filter is applied **Then** only records with that taxonomic_status are shown, with `provisional_name` visible in the list

Given a Tier 5 admin searching for "manombo" in the Species list **When** the search is submitted **Then** the record for "Bedotia sp. 'manombo'" appears (matched via `provisional_name`)

`species.ConservationAssessment`

```
list_display = [
    "species", "category", "source", "review_status",
    "assessment_date", "assessor", "flagged_by", "flagged_date",
]
list_filter = ["source", "review_status", "category"]
search_fields = ["species__scientific_name", "notes", "criteria"]
readonly_fields = ["flagged_date", "created_at"]
```

User stories:

As a Tier 3 conservation coordinator, **I want** to filter ConservationAssessments by `review_status = 'pending_review'`, **so that** I can work through all assessments needing attention.

Acceptance Criteria:

Given a Tier 3 coordinator with appropriate `is_staff` access **When** they filter by `review_status = 'pending_review'` **Then** only assessments with that status are shown across all species

Given a Tier 3 coordinator viewing a ConservationAssessment record with `review_status = 'pending_review'` **When** they change the status to `'accepted'` and save **Then** the record updates and disappears from the `pending_review` filter view

---

`populations.Institution`

```
list_display = [
    "name", "institution_type", "country", "city",
    "zims_member", "eaza_member", "aza_member",
]
list_filter = ["institution_type", "country", "zims_member", "eaza_member",
"aza_member"]
search_fields = ["name", "city", "country"]
```

`populations.ExSituPopulation`

```
list_display = [
    "species", "institution", "count_total", "count_male", "count_female",
    "breeding_status", "studbook_managed", "last_census_date",
]
list_filter = ["breeding_status", "studbook_managed", "institution"]
search_fields = ["species__scientific_name", "institution__name"]
inlines = [HoldingRecordInline]
```

`HoldingRecordInline` (TabularInline):

```
model = HoldingRecord
extra = 1
fields = ["date", "count_total", "count_male", "count_female", "count_unsexed",
"notes", "reporter"]
readonly_fields = ["reporter"]  # set automatically on save, not editable
```

User stories:

As a Tier 3 conservation coordinator, **I want** to record a new census count for a species held at my institution, **so that** the platform reflects current captive population data.

Acceptance Criteria:

Given a Tier 3 coordinator affiliated with "Cologne Zoo" accessing the ExSituPopulation admin **When** they view the population list **Then** they see ExSituPopulation records for all institutions (read access is global for Tier 3+)

Given the same Tier 3 coordinator attempting to save a HoldingRecord for an ExSituPopulation belonging to a different institution **When** they submit the form **Then** the save is rejected with a permission error (institution-scoped write enforcement)

Given a Tier 5 admin viewing an ExSituPopulation record **When** they open the inline HoldingRecord section **Then** all historical census records appear in chronological order, with the most recent first

---

`fieldwork.FieldProgram`

```
list_display = ["name", "lead_institution", "status", "region", "start_date"]
list_filter = ["status", "lead_institution"]
```

```
search_fields = ["name", "description", "region"]
filter_horizontal = ["focal_species", "partner_institutions"]
```

`accounts.User`

```
list_display = ["email", "name", "access_tier", "institution", "is_active",
"date_joined"]
list_filter = ["access_tier", "is_active", "institution"]
search_fields = ["email", "name"]
readonly_fields = ["date_joined", "last_login"]
fieldsets = [
    (None, {"fields": ["email", "name", "password"]}),
    ("Access", {"fields": ["access_tier", "institution", "is_active", "is_staff"]}),
    ("Profile", {"fields": ["expertise_areas", "orcid_id"]}),
    ("Dates", {"fields": ["date_joined", "last_login"]}),
]
```

Acceptance Criteria:

Given a Tier 5 admin elevating a user from Tier 2 to Tier 3 **When** they change `access_tier` to 3 and save **Then** the user's API requests immediately reflect Tier 3 access (token-based auth reads `access_tier` from the database on each request)

`integration.SyncJob`

```
list_display = ["job_type", "status", "started_at", "completed_at",
"records_processed", "records_updated", "records_skipped"]
list_filter = ["job_type", "status"]
readonly_fields = ["job_type", "status", "started_at", "completed_at",
"records_processed", "records_updated", "records_skipped", "error_log"]
```

SyncJob records are created and updated by Celery tasks, not by admin users. All fields are read-only in the admin.

`species.SpeciesLocality`

```
list_display = [
    "species", "locality_name", "locality_type", "presence_status",
    "water_body", "drainage_basin", "year_collected", "coordinate_precision",
]
list_filter = [
    "locality_type", "presence_status", "water_body_type",
    "coordinate_precision", "is_sensitive", "drainage_basin",
]
search_fields = [
    "locality_name", "water_body", "species__scientific_name",
    "source_citation", "collector",
]
```

```
readonly_fields = ["location_generalized", "drainage_basin_name", "created_at",
"updated_at"]
raw_id_fields = ["species", "drainage_basin"]
```

Uses `OSMGeoAdmin` (from `django.contrib.gis.admin`) for the `location` PointField, which provides a map-based point picker widget. This allows data curators to visually verify and place coordinates. `location_generalized` is read-only because it is auto-computed from `location` + `is_sensitive` on save.

Also register as an inline on the Species admin:

`SpeciesLocalityInline` (TabularInline):

```
model = SpeciesLocality
extra = 0
fields = [
    "locality_name", "locality_type", "presence_status",
    "water_body", "drainage_basin", "year_collected",
    "source_citation", "coordinate_precision", "is_sensitive",
]
readonly_fields = ["drainage_basin"]  # FK assigned by management command, not
manual entry
show_change_link = True  # link to full admin form for map-based coordinate editing
```

This inline shows all localities for a species when editing that species record, supporting the common workflow of reviewing all known localities for a given taxon.

User stories:

As a Tier 5 administrator adding a new field observation locality, **I want** to use a map widget to place the point coordinates, **so that** I can visually verify the location rather than manually entering lat/lng values.

Acceptance Criteria:

Given a Tier 5 admin creating a new SpeciesLocality record via the full admin form **When** they click on the `location` field's map widget **Then** an OpenStreetMap-based map renders; clicking on the map places a point marker and populates the coordinate fields

Given a Tier 5 admin editing a Species record **When** they view the SpeciesLocality inline section **Then** all existing locality records for that species are displayed; the "Add another" row allows creating a new locality; each row has a "change" link to the full SpeciesLocality admin form (for map-based coordinate editing)

`species.Watershed`

```
list_display = ["name", "pfafstetter_level", "pfafstetter_code", "area_sq_km"]
list_filter = ["pfafstetter_level"]
search_fields = ["name"]
readonly_fields = ["hybas_id", "geometry", "created_at"]
```

Watershed records are loaded via the `load_reference_layers` management command, not created through admin. Geometry is read-only. Metadata (name) can be edited in admin to improve naming.

`species.ProtectedArea`

```
list_display = ["name", "designation", "iucn_category", "status", "area_km2"]
list_filter = ["designation", "iucn_category", "status"]
search_fields = ["name"]
readonly_fields = ["wdpa_id", "geometry", "created_at"]
```

ProtectedArea records are loaded via the `load_reference_layers` management command. Geometry is read-only. Metadata can be edited in admin.

Technical Tasks

- Create `species/admin.py` (including SpeciesLocality with `OSMGeoAdmin`, Watershed, ProtectedArea, and SpeciesLocalityInline on SpeciesAdmin), `populations/admin.py`, `fieldwork/admin.py`, `accounts/admin.py`, `integration/admin.py` with registrations above
- Override `get_queryset()` on `ExSituPopulationAdmin` to enforce institution-scoped writes: Tier 3–4 users see all records but can only save changes to their own institution's records
- Set `AdminSite.site_header`, `AdminSite.site_title`, `AdminSite.index_title`
- Ensure `is_staff = True` is required for any Admin access; Tier 5 users must also have `is_superuser = True` for full access
- Write smoke tests: verify each ModelAdmin is registered and loads without error; verify list_display fields are accessible without N+1 queries (use `select_related` where needed)

Out of Scope

- Custom admin views or templates
- Custom admin actions (bulk import is post-MVP)
- Admin customization beyond registration configuration (e.g., custom widgets, AJAX lookups)
- Any frontend other than Django Admin

Gate Exit Criteria

Before marking Gate 04 complete:

1. All models load in Django Admin without errors
2. Institution-scoped write protection tests pass for ExSituPopulation
3. All `list_filter` values return correct filtered querysets
4. SpeciesLocality admin loads with OSMGeoAdmin map widget for the `location` field
5. SpeciesLocalityInline renders on the Species admin change form
6. Watershed and ProtectedArea admin pages load with read-only geometry fields
7. Invoke **@code-quality-reviewer** on admin files
8. Invoke **@security-reviewer** — this gate controls what authenticated coordinators can edit

---

Reconciliation

See [gate-04-reconciliation.md]() for the full spec-vs-implementation audit trail.